INFRASTRUCTURE & DEVOPS

Docker & Kubernetes

From Dockerfile to a locked-down customer cluster — images, Compose, Kubernetes objects, Helm, and the runbooks for when a pod will not start.

docker · kubernetes · containers · dockerfile · compose · helm · kubectl · k8s · devops · deployment · networking · security · troubleshooting · fde

Study Pack · Comprehensive reference & interview prep · ~72 min read · 15,806 words

CONTENTS

1. Why Containers Won
2. Images, Layers, Registries, Tags and Digests
3. Writing a Dockerfile Properly
4. Multi-Stage Builds, Slim Bases, and Non-Root
5. Running Containers
6. Docker Networking
7. Docker Compose for a Realistic Prototype
8. Debugging Containers
9. The Kubernetes Mental Model
10. Core Objects, With Worked YAML
11. kubectl Fluency
12. Scheduling and Reliability
13. Configuration and Packaging: Helm and Kustomize
14. Security in a Customer Cluster
15. Debugging a Broken Pod
16. Deploying Into a Customer's Environment
17. Interview Questions
18. Hands-On Lab Plan

Why Containers Won

Before containers, shipping software to a customer meant shipping a document. "Install Python 3.9, these 40 packages, this version of libssl, set these six environment variables, and make sure the locale is UTF-8." Half of it was wrong, the other half was undocumented tribal knowledge, and the first deployment call ran four hours. A container image replaces that document with an artifact: one file, one hash, byte-identical on your laptop and in the customer's data centre.

2013	~1	60s+	1
DOCKER'S FIRST PUBLIC RELEASE	SECOND TO START A CONTAINER	TO BOOT AN EQUIVALENT VM	KERNEL SHARED BY ALL CONTAINERS

What a container actually is

There is no "container" object in the Linux kernel. A container is an ordinary process that has been started with a particular arrangement of three long-standing kernel features:

NAMESPACES

Isolate what the process can *see*: PID (its own process tree, where your app is PID 1), mount (its own filesystem view), network (its own interfaces and ports), UTS (hostname), IPC, user (UID mapping), cgroup.

CGROUPS

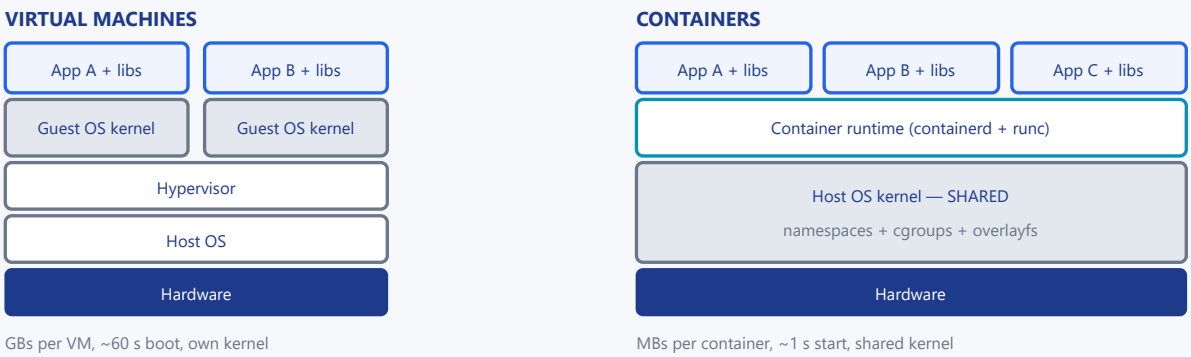
Limit what the process can *use*: CPU shares and quota, memory ceiling, block I/O weight, PIDs count. This is what makes `--memory=512m` enforceable rather than advisory.

UNION FILESYSTEM

Stack read-only image layers with a thin writable layer on top via overlayfs. Ten containers from one image share the same underlying bytes on disk until they write.

A VM virtualises hardware and runs a full guest kernel — gigabytes of image, tens of seconds to boot, strong isolation. A container shares the host kernel — megabytes, sub-second start, weaker isolation.

That kernel sharing has two consequences you will explain to customers repeatedly: a Linux container cannot run on a Windows kernel without a Linux VM underneath (which is exactly what Docker Desktop and WSL2 provide), and a container escape is a host compromise, which is why security teams care about what runs as root inside.



The structural difference: a duplicated kernel per workload versus one shared kernel with isolated views.

What this buys you on a customer engagement

Problem before containers	What the image gives you
"It works on my machine"	The same filesystem, libraries and interpreter everywhere; the only variables left are kernel, CPU architecture and injected configuration
Three days of dependency installation on a locked-down host	One `docker load` of a tarball you staged and checksummed
"Which version is in production?"	An immutable digest: `sha256:9f2c…`. There is exactly one answer
Prototype environment drifts from the demo environment	`compose.yaml` in the repo; `docker compose up` reproduces the whole stack in 30 seconds
Rollback means re-running the installer backwards	Deploy the previous tag; the old image is still in the registry
Security review of an unfamiliar binary	SBOM and CVE scan of the image, one artifact, one report

> **Docker is not the only runtime**
> Docker Engine is one implementation. Kubernetes dropped the Docker shim in 1.24 and talks to **containerd** or **CRI-O** directly. **Podman** is daemonless and rootless by default, and is the default on RHEL — `alias docker=podman` covers most commands. Images are standardised by the **OCI** spec, so an image built by Docker, Podman, Buildah, Kaniko or BuildKit runs anywhere. At a customer who has banned the Docker daemon for security reasons, this is the conversation that unblocks you.

Images, Layers, Registries, Tags and Digests

Understanding the layer model is what separates a 1.2 GB image that rebuilds in four minutes from a 90 MB image that rebuilds in eight seconds. It also explains why the secret you deleted in a later layer is still in the image.

The layer model

Each instruction in a Dockerfile that changes the filesystem (`RUN` , `COPY` , `ADD`) produces a new read-only layer containing only the *delta*. Metadata-only instructions (`ENV` , `WORKDIR` , `EXPOSE` , `LABEL` , `USER` , `CMD` , `ENTRYPOINT`) change the image config, not the filesystem. At runtime, overlayfs stacks the layers and adds a thin writable layer on top. Writing to a file that exists in a lower layer triggers a copy-up: the whole file is copied into the writable layer first.

BASH

```
docker history myapp:1.4.2
# IMAGE          CREATED BY                                      SIZE
# 9f2c1a3b…      CMD ["uvicorn" "app.main:app" …]                0B
# <missing>      USER appuser                                    0B
# <missing>      COPY . /app                                     12.4MB
# <missing>      RUN pip install -r requirements.txt             218MB
# <missing>      COPY requirements.txt /app/                     1.2kB
# <missing>      RUN apt-get update && apt-get install -y gcc    184MB
# <missing>      /bin/sh -c #(nop) CMD ["python3"]               0B

docker image inspect myapp:1.4.2 --format '{{json .RootFS.Layers}}' | jq -r '.[]'
docker image ls --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
docker system df            # images, containers, volumes, build cache - where the dis
```

Deleting a file does not shrink the image

`RUN wget secrets.tar && tar xf secrets.tar && rm secrets.tar` in three separate `RUN` instructions leaves the tarball permanently in layer 1. The later `rm` only writes a whiteout marker in a higher layer — anyone can `docker save` the image and extract it. The same applies to `ARG`-injected tokens, `.git` directories, and `~/.aws/credentials` copied in by an over-broad `COPY . .`. Fix it by doing the download, use and delete in **one** `RUN`, by using a multi-stage build, or by using BuildKit secret mounts.

Tags versus digests

TEXT

```
registry.corp.example.com:5000/platform/inference-api:1.4.2
└──────────── registry host:port ──────────┘ └─ path ─┘ └ tag ┘

registry.corp.example.com:5000/platform/inference-api@sha256:9f2c1a3b7e…
                                                      └──── digest ────┘
```

A tag is a mutable pointer. `:latest` is just a tag with a conventional name and no special meaning — nothing guarantees it is the newest. A digest is the SHA-256 of the image manifest: content-addressable, immutable, verifiable. If you pull by digest you provably get the same bytes every time.

Reference style	Use for	Risk
`:latest`	Local experimentation only	Unreproducible. Two nodes can run different code under the same tag
`:1.4.2`	Human-readable releases, Helm values	Mutable — a tag can be re-pushed unless the registry enforces immutability
`:1.4.2-a9f3c1d`	Release + git SHA; the practical default	Still technically mutable, but collisions are accidental not routine
`@sha256:…`	Production manifests, base images in Dockerfiles, anything audited	Unreadable; needs tooling to update. This is what it buys its safety with

BASH

```
docker inspect --format='{{index .RepoDigests 0}}' myapp:1.4.2
docker buildx imagetools inspect registry.corp/platform/api:1.4.2   # manifest list, p
crane digest registry.corp/platform/api:1.4.2                        # no daemon neede
skopeo inspect docker://registry.corp/platform/api:1.4.2             # works where Doc
```

Registries

BASH

```
docker login registry.corp.example.com:5000 -u svc_ci --password-stdin < /run/secrets/
docker tag myapp:1.4.2 registry.corp.example.com:5000/platform/myapp:1.4.2
docker push registry.corp.example.com:5000/platform/myapp:1.4.2
docker pull registry.corp.example.com:5000/platform/myapp@sha256:9f2c…

# Move an image between registries with no local daemon and no disk staging
skopeo copy docker://docker.io/library/postgres:16.2 \
            docker://registry.corp.example.com:5000/mirror/postgres:16.2
crane copy docker.io/library/redis:7.2 registry.corp:5000/mirror/redis:7.2

# Fully air-gapped: save to a file, carry it across, load on the far side
docker save registry.corp/platform/myapp:1.4.2 | gzip > myapp-1.4.2.tar.gz
sha256sum myapp-1.4.2.tar.gz > myapp-1.4.2.tar.gz.sha256
gunzip -c myapp-1.4.2.tar.gz | docker load
```

⚠ **Credentials land in plaintext by default**

`docker login` writes a base64 blob to `~/.docker/config.json`. On a shared bastion that is readable by anyone who can read your home directory. Use a credential helper (`docker-credential-pass`, `ecr-login`, `acr-env`), or log out when finished, or use short-lived registry tokens. Never put a registry password in a Dockerfile `ARG` — it is recorded in `docker history`.

Multi-architecture images

An Apple Silicon laptop builds `linux/arm64` by default; the customer's servers are almost certainly `linux/amd64`. The resulting `exec format error` wastes an afternoon if you do not expect it.

BASH

```
docker buildx create --name multi --use --bootstrap
docker buildx build --platform linux/amd64,linux/arm64 \
  -t registry.corp/platform/myapp:1.4.2 --push .
docker buildx imagetools inspect registry.corp/platform/myapp:1.4.2
# Name: …   MediaType: application/vnd.oci.image.index.v1+json
#   Platform: linux/amd64
#   Platform: linux/arm64

# Quick single-arch cross build when you just need amd64 from a Mac
docker build --platform linux/amd64 -t myapp:1.4.2 .
```

Writing a Dockerfile Properly

Most Dockerfiles in the wild are a transliteration of a README. A good one is engineered around two things the engine actually cares about: the order of layers for cache reuse, and what ends up in the final filesystem.

Instruction by instruction

Instruction	What it does	What people get wrong
`FROM`	Base image; starts a build stage	Using `:latest`; not pinning a digest for reproducible or audited builds
`RUN`	Executes a command, commits a layer	Splitting `apt-get update` and `install` into two `RUN`s — the cached update makes installs fail or install stale packages
`COPY`	Copies from build context into the image	`COPY . .` before dependency install, destroying all cache on every source edit
`ADD`	Like COPY, plus URL fetch and auto-extract of tars	Use `COPY` unless you specifically want tar extraction. `ADD <url>` hides a download with no checksum
`WORKDIR`	Sets cwd, creates the directory	Using `RUN cd /app` instead — the cd does not persist to the next instruction
`ENV`	Env var in image config, present at runtime	Putting secrets here. `docker inspect` prints them to anyone
`ARG`	Build-time variable only	Assuming it is secret. Values appear in `docker history`
`EXPOSE`	Documentation + a hint for `-P`	Believing it publishes the port. It does not; `-p` does
`USER`	UID/GID for subsequent instructions and runtime	Omitting it, so the container runs as root and fails the customer's admission policy
`ENTRYPOINT`	The fixed executable	Shell form, which wraps in `/bin/sh -c` and swallows SIGTERM
`CMD`	Default args to ENTRYPOINT, or the default command	Confusing it with ENTRYPOINT; `docker run img cmd` overrides CMD, not ENTRYPOINT
`HEALTHCHECK`	Periodic liveness probe for Docker	Expecting Kubernetes to use it — K8s ignores it and uses its own probes

ENTRYPOINT vs CMD, and the PID 1 signal problem

DOCKERFILE

```dockerfile
# exec form - argv array, no shell, signals reach your process
ENTRYPOINT ["/usr/local/bin/uvicorn"]
CMD ["app.main:app", "--host", "0.0.0.0", "--port", "8080"]

# shell form - runs as: /bin/sh -c "uvicorn app.main:app"
# sh becomes PID 1, forwards NOTHING, your app never sees SIGTERM
ENTRYPOINT uvicorn app.main:app        # avoid
```

With exec form, `docker run myapp --port 9090` replaces CMD and runs `uvicorn --port 9090`. With no ENTRYPOINT and only `CMD ["python","app.py"]`, `docker run myapp bash` gives you a shell — handy for debugging. Choose ENTRYPOINT when the image is a single-purpose tool, CMD-only when you want it to double as a general runtime.

> ⚠ **Zombie reaping**
>
> If your process spawns children and is PID 1, it must reap them or you accumulate zombies until the PID limit. Either handle `SIGCHLD` properly in the app, or add a tiny init: `docker run --init` (injects tini), or `ENTRYPOINT ["/sbin/tini","--","./app"]`, or in Kubernetes rely on the fact that most frameworks handle it — but verify with `ps -ef` inside the container under load.

Layer caching: the single highest-leverage idea

Docker reuses a cached layer when the instruction string is identical *and*, for `COPY` / `ADD`, the checksum of the copied files is unchanged. The moment one layer misses, every layer after it rebuilds. Therefore: order from least-frequently-changed to most-frequently-changed.

DOCKERFILE

```dockerfile
# BAD - every source edit reinstalls 200 MB of dependencies
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

DOCKERFILE

```
# GOOD — dependency layer survives source edits
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Same idea in every ecosystem: `COPY package.json package-lock.json ./` then `RUN npm ci` then `COPY . .`; `COPY go.mod go.sum ./` then `RUN go mod download` then `COPY . .`; `COPY pom.xml .` then `RUN mvn -B dependency:go-offline` then `COPY src ./src`.

DOCKERFILE

```
# One RUN, cleaned up in the same layer — the apt idiom
RUN apt-get update \

 && apt-get install -y --no-install-recommends \
      curl ca-certificates libpq5 \
 && rm -rf /var/lib/apt/lists/*
```

`--no-install-recommends` typically saves 50–150 MB. The `rm -rf /var/lib/apt/lists/*` must be in the same `RUN` or the cache files are already committed to a layer. On RHEL-family bases the equivalent is `microdnf install -y … && microdnf clean all`; on Alpine, `apk add --no-cache` does it in one flag.

BuildKit cache mounts and secrets

DOCKERFILE

```
# syntax=docker/dockerfile:1.7
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
# Persist the pip cache ACROSS builds without baking it into a layer
RUN --mount=type=cache,target=/root/.cache/pip \

    pip install -r requirements.txt

# A secret that is mounted during the RUN and never recorded in any layer
RUN --mount=type=secret,id=netrc,target=/root/.netrc \

    pip install --extra-index-url https://nexus.corp/simple internal-sdk==2.1.0
```

BASH

```
DOCKER_BUILDKIT=1 docker build --secret id=netrc,src=$HOME/.netrc -t myapp:1.4.2 .
```

Cache mounts are the difference between a 4-minute and a 25-second rebuild on a slow customer network. Secret mounts are the correct way to use a private package index token at build time without it appearing in `docker history`.

.dockerignore

The build context is tarred and sent to the daemon before the first instruction runs. A forgotten `.git` or `node_modules` can turn a 2 MB context into 900 MB, slow every build, and leak credentials.

TEXT

```
.git
.gitignore
**/node_modules
**/__pycache__
**/*.pyc
.venv
venv
.env
.env.*
*.log
dist
build
.pytest_cache
.mypy_cache
coverage/
.idea
.vscode
docs/
tests/fixtures/large/
*.tar.gz
Dockerfile*
docker-compose*.yml
README.md
.aws
.ssh
**/*.pem
**/*.key
```

BASH

```
du -sh .                                    # how big is your context?
docker build --no-cache --progress=plain .  # see "transferring context: N MB"
```

Multi-Stage Builds, Slim Bases, and Non-Root

A build toolchain is not a runtime dependency. Multi-stage builds let you compile with gcc, Maven or the full Node toolchain and then ship only the artifact. Typical result: 1.1 GB becomes 120 MB, and the CVE count drops by an order of magnitude because there is nothing left to be vulnerable.

Python — compile wheels, ship a venv

DOCKERFILE

```dockerfile
# syntax=docker/dockerfile:1.7
FROM python:3.12-slim AS builder
ENV PIP_DISABLE_PIP_VERSION_CHECK=1 PYTHONDONTWRITEBYTECODE=1
RUN apt-get update && apt-get install -y --no-install-recommends \

      build-essential libpq-dev gcc \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN --mount=type=cache,target=/root/.cache/pip \

    pip install -r requirements.txt

# ———————— runtime ————————
FROM python:3.12-slim AS runtime
RUN apt-get update && apt-get install -y --no-install-recommends \

      libpq5 curl ca-certificates \
 && rm -rf /var/lib/apt/lists/* \
 && groupadd -r app -g 10001 \
 && useradd -r -u 10001 -g app -d /app -s /sbin/nologin app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1
WORKDIR /app
COPY --chown=app:app ./app ./app
USER 10001
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \

  CMD curl -fsS http://127.0.0.1:8080/healthz || exit 1
ENTRYPOINT ["uvicorn"]
CMD ["app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

`gcc`, `build-essential` and `libpq-dev` exist only in the builder stage and never reach the final image — that alone removes roughly 400 MB and dozens of CVEs.

Go — static binary into scratch

DOCKERFILE

```
FROM golang:1.22 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download
COPY . .
RUN --mount=type=cache,target=/root/.cache/go-build \

    CGO_ENABLED=0 GOOS=linux go build -trimpath \
      -ldflags="-s -w -X main.version=${VERSION}" -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=builder /out/api /api
USER 65532:65532
EXPOSE 8080
ENTRYPOINT ["/api"]
```

Node — prune dev dependencies

DOCKERFILE

```
FROM node:20-bookworm-slim AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci

FROM node:20-bookworm-slim AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:20-bookworm-slim AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Choosing a base image

Base	Typical size	Shell?	libc	Use when
`ubuntu:22.04`	~78 MB	Yes	glibc	You need apt and familiarity; customer security baselines often mandate it
`debian:12-slim`	~75 MB	Yes	glibc	Sensible general default
`python:3.12-slim`	~130 MB	Yes	glibc	Python services; the safe default
`alpine:3.19`	~7 MB	Yes (ash)	**musl**	Tiny tools, Go/Rust statics — not for scientific Python
`distroless/base`	~20 MB	**No**	glibc	Hardened runtime, minimal CVE surface
`distroless/static`	~2 MB	**No**	none	Static Go/Rust binaries
`scratch`	0	No	none	Absolute minimum; you supply everything including CA certs
`ubi9-minimal`	~40 MB	Yes	glibc	Red Hat customers who require supported, certified base images

> ⚠ **Alpine and Python: measure before you commit**
> Alpine uses musl, not glibc, so manylinux wheels do not apply and pip compiles from source. A pandas/numpy/scipy image can take 15 minutes to build and end up *larger* than the slim variant, and musl's allocator has measurably worse performance for some workloads. For compiled languages Alpine is excellent; for the scientific Python stack, `-slim` usually wins on both size and build time.

Non-root, properly

DOCKERFILE

```
RUN groupadd -r app -g 10001 \

 && useradd -r -u 10001 -g app -d /app -s /sbin/nologin app
COPY --chown=app:app . /app
USER 10001:10001            # numeric UID, so Kubernetes runAsNonRoot can verify it
```

Use a numeric UID: `runAsNonRoot: true` in a Pod spec cannot resolve a username to a UID and will refuse to start the container with `CreateContainerConfigError: container has runAsNonRoot and image has non-numeric user`. Pick a high UID (10000+) to avoid colliding with host users. A non-root process cannot bind ports below 1024, so listen on 8080 and map it — in Kubernetes a Service maps 80 to 8080 transparently.

Scanning and SBOMs

BASH

```
trivy image --severity HIGH,CRITICAL registry.corp/platform/api:1.4.2
trivy image --ignore-unfixed --exit-code 1 --severity CRITICAL myapp:1.4.2   # CI gate
grype registry.corp/platform/api:1.4.2
docker scout cves myapp:1.4.2

syft myapp:1.4.2 -o spdx-json > sbom.spdx.json      # the customer WILL ask for this
trivy sbom sbom.spdx.json
cosign sign --key cosign.key registry.corp/platform/api:1.4.2
cosign verify --key cosign.pub registry.corp/platform/api:1.4.2
```

Base image updates fix most findings

The large majority of CVEs in a scan report come from the OS layer, not your code. Rebuilding weekly on a refreshed base often clears 80% of findings with zero code changes. Use `--ignore-unfixed` so the gate blocks on things you can actually act on, and keep a documented `.trivyignore` with an expiry date and a justification for each accepted risk — that file is what you hand to the customer's security reviewer.

Running Containers

The run-time flags are where theory meets the customer's host. Get ports, mounts, environment and limits right and most "it doesn't work here" problems never happen.

BASH

```
docker run -d \
  --name api \
  --restart unless-stopped \
  -p 127.0.0.1:8080:8080 \
  -e APP_ENV=prod \
  --env-file ./prod.env \
  -v api-data:/var/lib/app \
  -v /etc/ssl/corp-root.pem:/etc/ssl/certs/corp-root.pem:ro \
  --memory 2g --memory-reservation 1g \
  --cpus 1.5 \
  --pids-limit 256 \
  --read-only --tmpfs /tmp:rw,noexec,nosuid,size=128m \
  --security-opt no-new-privileges:true \
  --cap-drop ALL \
  --health-cmd 'curl -fsS http://127.0.0.1:8080/healthz || exit 1' \
  --health-interval 30s --health-timeout 3s --health-retries 3 \
  registry.corp/platform/api:1.4.2
```

Ports

Flag	Result
`-p 8080:8080`	Binds **0.0.0.0:8080** on the host — reachable from the whole network
`-p 127.0.0.1:8080:8080`	Loopback only. The right default on a shared or internet-facing host
`-p 8080:80`	Host 8080 to container 80
`-P`	Publish every `EXPOSE`d port to a random high host port
`--network host`	No network namespace; container uses the host stack directly. Linux only

Docker punches through the firewall

On Linux, Docker writes its own iptables rules in the `DOCKER` chain, which are evaluated *before* most ufw/firewalld rules. A container published with `-p 5432:5432` can be reachable from the internet even though `ufw status` says the port is denied. Always bind to `127.0.0.1` unless you deliberately intend public exposure, and tell the customer's security team what you published.

Volumes versus bind mounts

BASH

```
docker volume create api-data
docker run -v api-data:/var/lib/app …          # named volume: Docker-managed, portabl
docker run -v "$PWD/config:/etc/app:ro" …      # bind mount: host path, exact control
docker run --mount type=bind,src="$PWD/config",dst=/etc/app,readonly …   # explicit sy
docker run --tmpfs /tmp:rw,noexec,nosuid,size=64m …                      # RAM-backed

docker volume inspect api-data --format '{{.Mountpoint}}'
docker volume ls -f dangling=true
docker run --rm -v api-data:/data -v "$PWD:/backup" alpine \
  tar czf /backup/api-data-$(date +%F).tgz -C /data .
```

	Named volume	Bind mount
Location	`/var/lib/docker/volumes/…`	Any host path you choose
Permissions	Docker initialises from the image	Host UIDs apply directly — the usual source of permission errors
Portability	Works identically on any host	Depends on the host layout existing
Best for	Databases, persistent state	Local dev source mounting, injecting config and certs

> **Bind-mount permission denied**
> The container's UID must be able to read the host path. If the container runs as UID 10001 and the host directory is `0700 root:root`, it fails regardless of what is inside. Either `chown 10001` the host path, mount read-only and loosen to `0755`, or match UIDs with `--user "$(id -u):$(id -g)"`. On SELinux hosts add `:z` (shared) or `:Z` (private) to the mount, or the relabel never happens and every access is denied.

Restart policies and resources

Policy	Behaviour
`no`	Default. Never restarts
`on-failure[:5]`	Restart on non-zero exit, up to N times
`unless-stopped`	Always restart except when you explicitly stopped it — survives daemon restart. The usual choice
`always`	Restart even if you stopped it, after a daemon restart

BASH

```
docker stats --no-stream
# CONTAINER   CPU %    MEM USAGE / LIMIT     MEM %    NET I/O         PIDS
# api         48.21%   1.43GiB / 2GiB        71.52%   1.2GB / 840MB   38

docker update --memory 3g --cpus 2 api      # change limits on a running container
docker inspect api --format '{{.State.OOMKilled}}'   # true = it hit the memory limit
```

> ⚠ **Runtimes do not see cgroup limits automatically**
> A JVM before 10 (and without `-XX:+UseContainerSupport`) reads host RAM and sizes its heap for 64 GB inside a 2 GB container — then gets OOMKilled. Use `-XX:MaxRAMPercentage=75`. Node needs `--max-old-space-size`. Python libraries that size thread pools from `os.cpu_count()` will create 96 threads on a 1-CPU limit; set `OMP_NUM_THREADS`, `MKL_NUM_THREADS` and framework-specific equivalents explicitly.

Docker Networking

"The API container can't reach the database container" is the Compose-era version of the connectivity problem. Three facts resolve nearly all of it: which network each container is on, what name DNS resolves, and which port the process inside is actually bound to.

Driver	Behaviour	Use for
`bridge` (default)	Containers on `docker0`, **no DNS by name**, NAT to the host	Legacy; avoid for multi-container work
user-defined bridge	Same, plus an embedded DNS server resolving container and service names	Everything. This is what Compose creates
`host`	No network namespace; shares the host stack	Max throughput, port scanners, Linux only
`none`	Loopback only	Air-gapped testing, batch jobs with no network need
`macvlan`	Container gets its own MAC and an IP on the physical LAN	Legacy apps that must appear as a LAN host

BASH

```
docker network create --driver bridge appnet
docker network create --driver bridge --subnet 172.28.0.0/16 --gateway 172.28.0.1 appn
docker run -d --name db --network appnet postgres:16
docker run -d --name api --network appnet -e DB_HOST=db myapp:1.4.2
docker network inspect appnet --format '{{json .Containers}}' | jq
docker network connect othernet api      # attach a running container to a second netw
docker network disconnect appnet api
```

On a user-defined bridge, `db` resolves to the database container's IP from inside `api` — that is Docker's embedded DNS at `127.0.0.11`. It does **not** work on the default bridge, which is why beginners end up using the deprecated `--link`.

> **localhost inside a container is the container**
> `DB_HOST=localhost` in a container means "this container", not the host and not the database container. Use the service name on a shared network. To reach a service on the host machine, use `host.docker.internal` (Docker Desktop, and on Linux with `--add-host=host.docker.internal:host-gateway`) or the bridge gateway IP. And remember the port you connect to is the *container's* port, not the published host port: `api` talks to `db:5432` even if the host mapping is `15432:5432`.

BASH

```
# Debug from inside, without installing anything in the app image
docker run --rm -it --network appnet nicolaka/netshoot bash
  dig db
  nc -zv db 5432
  curl -v http://api:8080/healthz
  ss -tulpn

# Or join the target container's network namespace directly
docker run --rm -it --network container:api nicolaka/netshoot ss -tulpn
```

That last form is the one to remember: it puts a fully-equipped debug container inside the *exact* network namespace of your minimal distroless app container, so `ss`, `tcpdump` and `curl` all see what the app sees — without adding a single byte to the production image.

Docker Compose for a Realistic Prototype

Compose is how you hand a customer a working stack. One file, one command, and their engineer has the whole system running on a laptop — API, database, vector store, cache and UI. On a two-week engagement this is often what you actually ship first.

YAML

```yaml
# compose.yaml — RAG prototype: UI + API + Postgres + Qdrant + Redis
name: ragdemo

x-logging: &default-logging
  driver: json-file
  options:
    max-size: "20m"
    max-file: "5"

services:
  api:
    build:
      context: ./api
      dockerfile: Dockerfile
      target: runtime
      args:
        VERSION: ${VERSION:-dev}
    image: registry.corp/platform/rag-api:${VERSION:-dev}
    restart: unless-stopped
    env_file: [.env]
    environment:
      DATABASE_URL: postgresql://app:${POSTGRES_PASSWORD}@db:5432/ragdb
      QDRANT_URL: http://vectordb:6333
      REDIS_URL: redis://cache:6379/0
      LOG_LEVEL: ${LOG_LEVEL:-info}
      NO_PROXY: localhost,127.0.0.1,db,vectordb,cache,.internal
    ports:
      - "127.0.0.1:8080:8080"
    depends_on:
      db:        { condition: service_healthy }
      vectordb:  { condition: service_healthy }
      cache:     { condition: service_started }
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://127.0.0.1:8080/healthz"]
      interval: 15s
      timeout: 3s
      retries: 5
      start_period: 30s
    deploy:
      resources:
        limits: { cpus: "2.0", memory: 2G }
    logging: *default-logging
    networks: [backend, frontend]

  db:
    image: postgres:16.2-alpine
    restart: unless-stopped
    environment:
```

```
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?POSTGRES_PASSWORD must be set}
      POSTGRES_DB: ragdb
      PGDATA: /var/lib/postgresql/data/pgdata
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d ragdb"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
    command:
      - "postgres"
      - "-c"
      - "shared_preload_libraries=vector"
      - "-c"
      - "max_connections=200"
    logging: *default-logging
    networks: [backend]

  vectordb:
    image: qdrant/qdrant:v1.9.0
    restart: unless-stopped
    volumes:
      - qdrantdata:/qdrant/storage
    healthcheck:
      test: ["CMD-SHELL", "bash -c '>/dev/tcp/127.0.0.1/6333'"]
      interval: 10s
      timeout: 3s
      retries: 10
      start_period: 15s
    logging: *default-logging
    networks: [backend]

  cache:
    image: redis:7.2-alpine
    restart: unless-stopped
    command: ["redis-server", "--save", "60", "1", "--maxmemory", "512mb",
              "--maxmemory-policy", "allkeys-lru"]
    volumes:
      - redisdata:/data
    logging: *default-logging
    networks: [backend]

  ui:
    build: ./ui
    restart: unless-stopped
```

```
    environment:
      VITE_API_BASE: http://localhost:8080
    ports:
      - "127.0.0.1:3000:3000"
    depends_on:
      api: { condition: service_healthy }
    logging: *default-logging
    networks: [frontend]

  migrate:
    image: registry.corp/platform/rag-api:${VERSION:-dev}
    profiles: ["tools"]
    env_file: [.env]
    environment:
      DATABASE_URL: postgresql://app:${POSTGRES_PASSWORD}@db:5432/ragdb
    command: ["alembic", "upgrade", "head"]
    depends_on:
      db: { condition: service_healthy }
    networks: [backend]
    restart: "no"

volumes:
  pgdata:
  qdrantdata:
  redisdata:

networks:
  backend:
    driver: bridge
  frontend:
    driver: bridge
```

BASH

```
docker compose up -d                       # build if needed, start everything
docker compose up -d --build               # force rebuild
docker compose ps                          # status + health + published ports
docker compose logs -f api                 # follow one service
docker compose logs --since 10m --tail 100 # bounded
docker compose exec api bash               # shell in a running service
docker compose run --rm migrate            # one-off task from the "tools" profile
docker compose config                      # render the fully-resolved file - debug en
docker compose down                        # stop and remove containers + networks
docker compose down -v                     # ALSO delete volumes - destroys the databa
docker compose restart api
docker compose pull && docker compose up -d   # roll to new images
```

✦ **Three details that make a Compose file demo-proof**

First, `depends_on` with `condition: service_healthy` — plain `depends_on` only waits for the container to *start*, so your API still races the database and crashes on first boot. Second, `${VAR:?message}` so a missing secret fails immediately with a clear error instead of starting Postgres with a blank password. Third, the `logging` anchor — without a size cap, a chatty service fills the customer's laptop disk during a long demo.

BASH

```bash
# Layering an override for the customer's environment
# compose.yaml (base, committed)  +  compose.override.yaml (local, auto-loaded)
docker compose -f compose.yaml -f compose.prod.yaml up -d
docker compose --profile tools run --rm migrate
docker compose --env-file .env.staging config   # verify before you run it on their ho
```

⚠ **Compose is not a production platform**

It has no rescheduling, no rolling update, no multi-host story. It is excellent for prototypes, demos, CI environments and single-box edge installs. The moment the customer asks about high availability, zero-downtime deploys or horizontal scaling, that is the Kubernetes conversation — and the Compose file you already wrote is the specification you translate from.

Debugging Containers

A container is a process with a restricted view. Every debugging technique reduces to one of: read its output, get inside its namespaces, or read its configuration from the outside.

BASH

```
docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
docker logs --tail 200 --timestamps api
docker logs -f --since 5m api
docker inspect api | jq '.[0].State'
docker inspect api --format '{{.State.ExitCode}} {{.State.Error}} {{.State.OOMKilled}}
docker exec -it api sh                      # /bin/sh exists more often than bash
docker exec -u 0 -it api sh                 # as root, when you need to install toolin
docker top api                              # processes, mapped to HOST pids
docker diff api                             # files changed vs the image - revealing
docker stats --no-stream api
docker cp api:/app/config.yaml ./           # pull a file out for inspection
docker events --since 10m                   # the daemon's own timeline
```

Exit codes

Code	Meaning	First thing to check
0	Clean exit	A foreground process that finished — your CMD probably daemonised itself
1	Generic application error	`docker logs`; usually a config or connection failure at startup
125	Docker itself failed	Bad flag, invalid mount, unknown network
126	Command found but not executable	Missing `chmod +x`, or a CRLF line ending in an entrypoint script
127	Command not found	Wrong path, or the binary needs a shell/libc that distroless lacks
137	128+9, SIGKILL	**OOMKilled** — check `.State.OOMKilled`; or a stop timeout expired
139	128+11, SIGSEGV	Native crash; often an architecture mismatch or a bad shared library
143	128+15, SIGTERM	Normal stop — the app shut down when asked

Debugging an image that will not start at all

BASH

```
docker run --rm -it --entrypoint sh myapp:1.4.2        # bypass ENTRYPOINT, look aroun
docker run --rm -it --entrypoint sh myapp:1.4.2 -c 'ls -la /app; id; env | sort'
docker create --name tmp myapp:1.4.2 && docker export tmp | tar -tv | head -50
docker run --rm -it --network container:api nicolaka/netshoot   # debug its network
docker run --rm -it -v api-data:/data alpine ls -la /data       # inspect a volume
dive myapp:1.4.2                                        # layer-by-layer size explorer
```

"It works on my machine" — the real failure modes

ARCHITECTURE MISMATCH

Built `arm64` on a Mac, ran on `amd64`. Symptom: `exec format error` or `exec /app: no such file or directory` on a binary that clearly exists. Fix: `docker buildx build --platform linux/amd64`. Check with `docker image inspect --format '{{.Architecture}}'`.

STALE BUILD CACHE

A `RUN apt-get update` layer cached from last month installs stale packages, or a `COPY` cache hit hides a changed file. Confirm with `docker build --no-cache`.

MISSING RUNTIME LIBRARY

Works in the builder stage, fails in runtime because `libpq5` was never installed in the final stage. Symptom: `ImportError: libpq.so.5: cannot open shared object file`. Check with `ldd` inside the image.

ENV PRESENT LOCALLY, ABSENT IN THE IMAGE

Your shell exported `OPENAI_API_KEY`; the container never received it. `docker exec api env | sort` is the one-command check.

BOUND TO 127.0.0.1 INSIDE

The app listens on loopback inside its own namespace, so the published port connects to nothing. It must bind `0.0.0.0`. Verify with `docker exec api ss -tulpn`.

FILE PERMISSIONS AFTER USER

`COPY` without `--chown` leaves files owned by root; the app runs as UID 10001 and cannot write its cache directory. `docker exec -u 10001 api touch /app/cache/x` reproduces it instantly.

CORPORATE CA MISSING

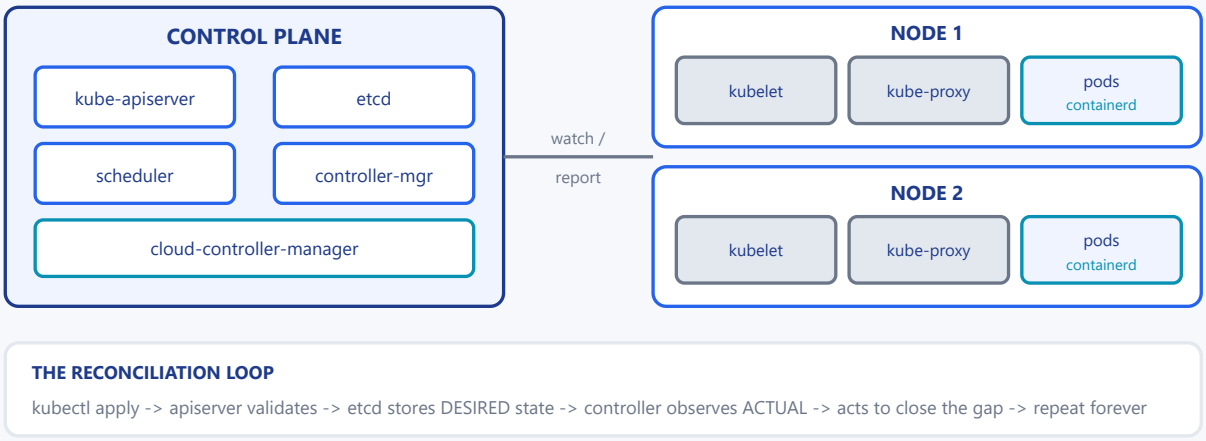
Outbound HTTPS fails with "unknown authority" because the image trusts public roots only. Copy the corporate root into the image or mount it, and set `REQUESTS_CA_BUNDLE` / `NODE_EXTRA_CA_CERTS` / `SSL_CERT_FILE`.

CLOCK AND TIMEZONE

Containers inherit the host clock but default to UTC. Signature validation and JWT expiry fail if the host clock drifts. `docker exec api date -u` versus the host.

The Kubernetes Mental Model

Kubernetes is a control loop over a declarative API. You write down the state you want; a set of controllers continuously compares that to reality and acts to close the gap. Every object, every failure mode and every debugging step makes sense once you hold that one idea firmly.



You never create a container. You record intent; controllers make reality match it, continuously.

The components

Component	Role	When it is the problem
`kube-apiserver`	The only thing that talks to etcd; REST API, authn/authz, admission	`kubectl` hangs or returns 401/403; webhook timeouts block every apply
`etcd`	Consistent key-value store of all cluster state	Slow etcd = slow everything. Disk latency is the usual culprit
`kube-scheduler`	Assigns pending pods to nodes based on resources, affinity, taints	Pods stuck `Pending` with "0/5 nodes are available"
`controller-manager`	Runs the loops: Deployment, ReplicaSet, Node, Job, endpoints	Replicas never created; endpoints never populated
`kubelet`	Node agent: pulls images, starts containers, runs probes, reports status	`NotReady` node, `ImagePullBackOff`, probe failures
`kube-proxy` / CNI	Service VIP routing (iptables/IPVS) and pod networking	Service IP unreachable while the pod IP works
CoreDNS	Cluster DNS for Services	Name resolution fails cluster-wide; check it first in any networking incident

Why it exists (the honest version)

Kubernetes solves scheduling across many machines, self-healing when a node dies, rolling updates with health gating, service discovery and load balancing, declarative configuration with rollback, and horizontal autoscaling — all behind one API with a consistent extension model. That is a genuinely large amount of operational work it removes.

> ⚠ **And when it is the wrong answer**
> A single-service prototype on one VM does not need a cluster; systemd or Compose is less to run, less to secure and less to explain. If the customer has no platform team, introducing Kubernetes transfers an operational burden they cannot carry after you leave. The FDE judgement is to match the platform to what the customer can actually operate on their own — and to say so out loud even when they asked for Kubernetes by name.

Core Objects, With Worked YAML

You will spend most of your Kubernetes time with about ten object types. Here is each one, what it is for, and YAML you can adapt rather than invent.

Pod — the unit of scheduling

A Pod is one or more containers that share a network namespace (same IP, same localhost, same port space) and can share volumes. You rarely create one directly; you create a Deployment that creates them. But understanding the Pod spec is essential because every higher object embeds it as `template`.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: debug-shell
  namespace: ragdemo
  labels: { app: debug }
spec:
  terminationGracePeriodSeconds: 5
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    fsGroup: 10001
    seccompProfile: { type: RuntimeDefault }
  containers:
    - name: shell
      image: nicolaka/netshoot:latest
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
      resources:
        requests: { cpu: 50m, memory: 64Mi }
        limits:   { cpu: 200m, memory: 256Mi }
      volumeMounts:
        - { name: tmp, mountPath: /tmp }
  volumes:
    - name: tmp
      emptyDir: { sizeLimit: 64Mi }
```

Multi-container patterns worth knowing: a **sidecar** runs alongside (log shipper, proxy); an **init container** runs to completion before app containers start (schema migration, waiting for a dependency, fetching config); an **ambassador** proxies outbound connections. Containers in a Pod are always co-scheduled on one node — that is the defining property.

Deployment — what you actually write

Deployment manages ReplicaSets; a ReplicaSet manages Pods. On an update, the Deployment creates a new ReplicaSet and shifts replicas across according to the rollout strategy, keeping old ReplicaSets around for rollback.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rag-api
  namespace: ragdemo
  labels: { app.kubernetes.io/name: rag-api, app.kubernetes.io/version: "1.4.2" }
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # at most 1 extra pod above replicas during rollout
      maxUnavailable: 0    # never drop below 3 ready - zero-downtime
  selector:
    matchLabels: { app: rag-api }      # IMMUTABLE after creation
  template:
    metadata:
      labels: { app: rag-api }
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
        checksum/config: "{{ sha256sum of the ConfigMap }}"   # forces rollout on conf
    spec:
      serviceAccountName: rag-api
      imagePullSecrets:
        - name: corp-registry
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        fsGroup: 10001
        seccompProfile: { type: RuntimeDefault }
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels: { app: rag-api }
      initContainers:
        - name: wait-for-db
          image: busybox:1.36
          command: ['sh','-c','until nc -z postgres 5432; do echo waiting; sleep 2; do
      containers:
        - name: api
          image: registry.corp/platform/rag-api@sha256:9f2c1a3b7e4d5c6a8b9f0e1d2c3b4a5
          imagePullPolicy: IfNotPresent
          ports:
            - { name: http, containerPort: 8080, protocol: TCP }
```

```yaml
      env:
        - name: LOG_LEVEL
          valueFrom: { configMapKeyRef: { name: rag-api-config, key: log_level } }
        - name: DATABASE_PASSWORD
          valueFrom: { secretKeyRef: { name: rag-api-secrets, key: db_password } }
        - name: POD_NAME
          valueFrom: { fieldRef: { fieldPath: metadata.name } }
        - name: NODE_NAME
          valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
      envFrom:
        - configMapRef: { name: rag-api-config }
      resources:
        requests: { cpu: 250m, memory: 512Mi }
        limits:   { cpu: "2",  memory: 2Gi }
      startupProbe:
        httpGet: { path: /healthz, port: http }
        failureThreshold: 30
        periodSeconds: 5          # allows up to 150 s to start
      readinessProbe:
        httpGet: { path: /readyz, port: http }
        initialDelaySeconds: 5
        periodSeconds: 10
        timeoutSeconds: 2
        failureThreshold: 3
      livenessProbe:
        httpGet: { path: /healthz, port: http }
        periodSeconds: 20
        timeoutSeconds: 3
        failureThreshold: 3
      lifecycle:
        preStop:
          exec: { command: ["sh","-c","sleep 5"] }   # let endpoints drain first
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: ["ALL"] }
      volumeMounts:
        - { name: cache, mountPath: /app/.cache }
        - { name: tmp,   mountPath: /tmp }
        - { name: corp-ca, mountPath: /etc/ssl/certs/corp-root.pem, subPath: corp-
    volumes:
      - name: cache
        emptyDir: { sizeLimit: 512Mi }
      - name: tmp
        emptyDir: { sizeLimit: 128Mi }
      - name: corp-ca
        configMap: { name: corp-ca-bundle }
    terminationGracePeriodSeconds: 45
```

The preStop sleep is not a hack

When a Pod is deleted, two things happen in parallel: the kubelet sends SIGTERM, and the endpoints controller removes the Pod from Service endpoints. The second propagates to every kube-proxy asynchronously, so for a second or two traffic still arrives at a shutting-down Pod. A short `preStop` sleep plus a graceful shutdown in the app removes the 502 spike that customers notice during every deploy.

Service — stable networking

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: rag-api
  namespace: ragdemo
spec:
  type: ClusterIP                # default: a virtual IP reachable only inside the cl
  selector: { app: rag-api }     # matches POD labels, not the Deployment name
  ports:
    - name: http
      port: 80                   # the Service port
      targetPort: http           # the container port (name or number)
      protocol: TCP
---

apiVersion: v1
kind: Service
metadata:
  name: rag-api-headless
spec:
  clusterIP: None                # headless: DNS returns pod IPs directly, no VIP
  selector: { app: rag-api }
  ports: [{ port: 8080 }]
```

Type	What it creates	Reachable from	Use for
`ClusterIP`	A virtual IP + DNS name	Inside the cluster only	Nearly everything
`NodePort`	Opens the same high port (30000–32767) on *every* node	Anyone who can reach a node	Bare metal without a load balancer; quick demos
`LoadBalancer`	NodePort + asks the cloud provider for an external LB	The internet or an internal VNet	Cloud clusters; one LB per Service gets expensive
`ExternalName`	A CNAME record, no proxying	n/a	Pointing an in-cluster name at an external database
Headless	DNS A records for each Pod	Inside the cluster	StatefulSets, client-side load balancing, gRPC

Service DNS follows a fixed pattern: `<service>.<namespace>.svc.cluster.local`. Inside the same namespace, `rag-api` is enough; across namespaces you need `rag-api.ragdemo`. This is the single most common cause of "connection refused" in a multi-namespace install.

Ingress — HTTP routing from outside

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: rag
  namespace: ragdemo
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "25m"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "120"
    cert-manager.io/cluster-issuer: corp-ca-issuer
spec:
  ingressClassName: nginx
  tls:
    - hosts: [rag.apps.corp.example.com]
      secretName: rag-tls
  rules:
    - host: rag.apps.corp.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: rag-api
                port: { number: 80 }
          - path: /
            pathType: Prefix
            backend:
              service:
                name: rag-ui
                port: { number: 80 }
```

An Ingress object is inert without an Ingress *controller* (ingress-nginx, Traefik, HAProxy, or a cloud ALB controller) actually running in the cluster. Annotation names are controller-specific and do not port across. The newer Gateway API is the standards-track replacement with better multi-tenancy, but ingress-nginx is what you will find installed at most customers today.

ConfigMap and Secret

YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: rag-api-config, namespace: ragdemo }
data:
  log_level: "info"
  max_tokens: "2048"
  NO_PROXY: "localhost,127.0.0.1,.svc,.cluster.local,10.0.0.0/8"
  app.yaml: |
    retriever:
      top_k: 8
      rerank: true
    limits:
      request_timeout_s: 30
---

apiVersion: v1
kind: Secret
metadata: { name: rag-api-secrets, namespace: ragdemo }
type: Opaque
stringData:                       # stringData is plaintext at apply; data is base64
  db_password: "change-me-in-a-real-cluster"
  openai_api_key: "sk-..."
```

BASH

```bash
kubectl create configmap rag-api-config --from-file=app.yaml --from-literal=log_level=
  --dry-run=client -o yaml | kubectl apply -f -
kubectl create secret generic rag-api-secrets --from-env-file=.env.prod
kubectl create secret docker-registry corp-registry \
  --docker-server=registry.corp.example.com:5000 \
  --docker-username=svc_k8s --docker-password="$REG_PASS"
kubectl get secret rag-api-secrets -o jsonpath='{.data.db_password}' | base64 -d
```

Kubernetes Secrets are base64, not encryption

Anyone with `get secret` RBAC in the namespace, or read access to etcd, can recover the plaintext. Minimum bar: enable encryption at rest on the apiserver, restrict Secret RBAC tightly, and never commit Secret manifests to git. Better: External Secrets Operator or the Secrets Store CSI driver pulling from Vault, AWS Secrets Manager or Azure Key Vault, so the cluster holds a reference rather than the value.

Mounted ConfigMaps update in-place (within a minute or so) but environment variables from them do **not** — the process must restart. The `checksum/config` annotation pattern in the Deployment template

forces a rolling restart whenever the ConfigMap changes, which is how Helm charts solve this.

Job and CronJob

YAML

```yaml
apiVersion: batch/v1
kind: Job
metadata: { name: db-migrate-1-4-2, namespace: ragdemo }
spec:
  backoffLimit: 3
  activeDeadlineSeconds: 900
  ttlSecondsAfterFinished: 3600     # auto-clean the Job object
  template:
    spec:
      restartPolicy: Never          # or OnFailure; NEVER "Always" for a Job
      containers:
        - name: migrate
          image: registry.corp/platform/rag-api:1.4.2
          command: ["alembic", "upgrade", "head"]
          envFrom:
            - secretRef: { name: rag-api-secrets }
---

apiVersion: batch/v1
kind: CronJob
metadata: { name: reindex, namespace: ragdemo }
spec:
  schedule: "30 2 * * *"
  timeZone: "Asia/Kolkata"          # Kubernetes 1.27+
  concurrencyPolicy: Forbid         # Allow | Forbid | Replace
  startingDeadlineSeconds: 300
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: reindex
              image: registry.corp/platform/rag-api:1.4.2
              command: ["python", "-m", "jobs.reindex"]
              resources:
                requests: { cpu: 500m, memory: 1Gi }
                limits:   { cpu: "2",  memory: 4Gi }
```

PersistentVolumeClaim

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: qdrant-data, namespace: ragdemo }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3            # omit to use the cluster default
  resources:
    requests: { storage: 100Gi }
```

Access mode	Meaning	Typical backing
`ReadWriteOnce`	Mounted read-write by one *node*	EBS, Azure Disk, GCE PD, most block storage
`ReadWriteOncePod`	Exactly one Pod (1.22+)	Strict single-writer databases
`ReadOnlyMany`	Read-only by many nodes	Shared reference data
`ReadWriteMany`	Read-write by many nodes	NFS, EFS, Azure Files, CephFS — not available on plain block storage

For stateful workloads use a StatefulSet rather than a Deployment: it gives stable network identities (`db-0` , `db-1`), ordered rollout, and a `volumeClaimTemplates` block that gives each replica its own PVC. Note that deleting a StatefulSet does *not* delete its PVCs — by design, and a frequent source of orphaned storage cost at customers.

Namespace and ResourceQuota

YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: ragdemo
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
---

apiVersion: v1
kind: ResourceQuota
metadata: { name: ragdemo-quota, namespace: ragdemo }
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    persistentvolumeclaims: "6"
    count/pods: "40"
---

apiVersion: v1
kind: LimitRange
metadata: { name: ragdemo-defaults, namespace: ragdemo }
spec:
  limits:
    - type: Container
      default:        { cpu: 500m, memory: 512Mi }
      defaultRequest: { cpu: 100m, memory: 128Mi }
      max:            { cpu: "4",  memory: 8Gi }
```

⚠ **A quota makes requests mandatory**

Once a ResourceQuota sets `requests.cpu`, every Pod in that namespace must declare requests or it is rejected outright with `failed quota: must specify cpu,memory`. A LimitRange supplies defaults so existing manifests keep working. Customers almost always have quotas on shared clusters — assume they do and set requests from the start.

kubectl Fluency

Twenty-five commands cover essentially all day-to-day work. Learn the output formatting flags too — `-o jsonpath` and `-o custom-columns` turn kubectl into a data source you

can script against.

Contexts and kubeconfig

BASH

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context acme-prod
kubectl config set-context --current --namespace=ragdemo    # stop typing -n everywher
kubectl config view --minify --raw                           # the ACTIVE context, wit

export KUBECONFIG=~/.kube/acme-prod.yaml                     # one file per customer
export KUBECONFIG=~/.kube/config:~/.kube/acme.yaml           # merge several
kubectl cluster-info
kubectl version --short
kubectl auth can-i create deployments -n ragdemo
kubectl auth can-i --list -n ragdemo                         # everything you are allo
```

Make the context visible

Running `kubectl delete` against the wrong cluster is a career event. Put the context and namespace in your shell prompt (kube-ps1, starship) and keep each customer's kubeconfig in a separate file selected by an explicit `export KUBECONFIG=` — never merged into your default config where a stray `use-context` can bite you.

The commands that matter

BASH

```bash
kubectl get pods -o wide                        # adds node, pod IP, nominated node
kubectl get pods -w                             # watch changes live
kubectl get pods -A                             # all namespaces
kubectl get pods -l app=rag-api                 # by label selector
kubectl get all -n ragdemo                      # common types (NOT literally everythin
kubectl get events --sort-by=.lastTimestamp -n ragdemo | tail -30
kubectl describe pod rag-api-7d4f8b6c9-xk2mq    # spec + status + EVENTS. Read the bott
kubectl logs rag-api-7d4f8b6c9-xk2mq -f
kubectl logs -l app=rag-api --tail=100 --prefix --all-containers
kubectl logs rag-api-7d4f8b6c9-xk2mq --previous    # the CRASHED container's output
kubectl exec -it rag-api-7d4f8b6c9-xk2mq -- sh
kubectl exec rag-api-7d4f8b6c9-xk2mq -c api -- env | sort
kubectl port-forward svc/rag-api 8080:80
kubectl port-forward pod/rag-api-7d4f8b6c9-xk2mq 8080:8080
kubectl apply -f manifests/ --recursive
kubectl apply -k overlays/prod                  # kustomize
kubectl diff -f deploy.yaml                     # what WOULD change - run this first
kubectl rollout status deploy/rag-api --timeout=120s
kubectl rollout history deploy/rag-api
kubectl rollout undo deploy/rag-api --to-revision=3
kubectl rollout restart deploy/rag-api          # re-roll without changing the spec
kubectl scale deploy/rag-api --replicas=5
kubectl set image deploy/rag-api api=registry.corp/platform/rag-api:1.4.3
kubectl top pods --sort-by=memory               # needs metrics-server
kubectl top nodes
kubectl cp ragdemo/rag-api-xk2mq:/app/dump.json ./dump.json
kubectl debug -it rag-api-xk2mq --image=nicolaka/netshoot --target=api   # ephemeral c
kubectl explain deployment.spec.strategy.rollingUpdate   # built-in schema docs
kubectl api-resources | grep -i ingress         # what types exist, short names, group
```

Output formatting

BASH

```
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.phase}{"
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}' | tr ' ' '\n' | so
kubectl get pod rag-api-xk2mq -o jsonpath='{.status.containerStatuses[0].restartCount}
kubectl get nodes -o custom-columns=\
'NAME:.metadata.name,READY:.status.conditions[-1].type,CPU:.status.capacity.cpu,MEM:.s
kubectl get deploy -A -o json | jq -r '
  .items[] | select(.spec.template.spec.containers[].resources.limits == null)
  | "\(.metadata.namespace)/\(.metadata.name) has no limits"'
kubectl get pods --field-selector status.phase≠Running,status.phase≠Succeeded -A
kubectl get pods --sort-by='.status.containerStatuses[0].restartCount' -A | tail -20
kubectl get secret rag-api-secrets -o go-template='{{range $k,$v := .data}}{{$k}}={{$v
```

describe vs logs vs events

Tool	Answers	Use when
`describe`	What the control plane thinks: scheduling decisions, image pulls, probe results, mount failures	The Pod never started, or is restarting. The **Events** block at the bottom is the answer 80% of the time
`logs`	What the application printed	The container started and then misbehaved. Add `--previous` after a crash
`get events`	Cluster-wide timeline including node pressure, evictions, scheduler messages	Several things broke at once, or the Pod object was deleted before you could describe it

> **Events expire**
> Default retention is one hour. If a customer reports a problem from last night, the events are gone — you need the logs (if the Pod object still exists), the application's own logging backend, or `kubectl get pods -o yaml` for `lastState.terminated`, which preserves the exit code and reason of the previous container run.

Scheduling and Reliability

Requests and limits are the two numbers that determine whether your workload gets scheduled, whether it gets throttled, and whether it gets killed first when a node runs out

of memory. Most production incidents in a customer cluster trace back to these being wrong or absent.

Requests versus limits

YAML

```yaml
resources:
  requests:            # used by the SCHEDULER to pick a node; a reservation
    cpu: 250m          # 0.25 of a core
    memory: 512Mi
  limits:              # enforced by the KERNEL via cgroups at runtime
    cpu: "2"
    memory: 2Gi
```

CPU is compressible: exceeding the limit causes throttling (the process is paused within each 100 ms period), not termination. Memory is not compressible: exceeding the limit causes an immediate OOM kill with exit code 137. That asymmetry drives the standard advice — set memory request equal to memory limit, and be generous or omit CPU limits entirely if latency matters more than noisy-neighbour fairness.

QoS class	Condition	Eviction order under node pressure
`Guaranteed`	Every container has requests == limits for both CPU and memory	Evicted **last**
`Burstable`	At least one request set, but not matching limits	Evicted second, worst-offender-over-request first
`BestEffort`	No requests or limits at all	Evicted **first**, always

BASH

```bash
kubectl get pod rag-api-xk2mq -o jsonpath='{.status.qosClass}'
kubectl describe node ip-10-0-3-21 | sed -n '/Allocated resources/,/^Events/p'
# Resource  Requests      Limits
# cpu       3250m (81%)   9 (225%)       ← overcommitted on limits; fine, but watch t
# memory    7409Mi (62%)  16Gi (137%)
kubectl top pod --containers -n ragdemo
```

> ⚠ **CPU limits cause latency you will be blamed for**
> A pod with `limits.cpu: "1"` that briefly needs 1.5 cores is throttled — the kernel freezes it for the rest of the 100 ms period. The symptom is p99 latency spikes with CPU utilisation that looks comfortable. Check `container_cpu_cfs_throttled_seconds_total` in Prometheus. For latency-sensitive services, many teams set requests carefully and omit CPU limits entirely.

Probes

Probe	On failure	Purpose	Get wrong and…
`startupProbe`	Kills the container	Protects slow starters; disables the other two until it passes	Without it, a 90-second JVM start is killed by liveness at 30 s, forever
`readinessProbe`	Removes the Pod from Service endpoints	Traffic gating — no restart	Too aggressive = flapping endpoints and intermittent 503s
`livenessProbe`	**Restarts** the container	Recover from deadlock only	Pointing it at a dependency turns a database blip into a cluster-wide restart storm

YAML

```yaml
startupProbe:    { httpGet: {path: /healthz, port: 8080}, failureThreshold: 30, period
readinessProbe:  { httpGet: {path: /readyz,  port: 8080}, periodSeconds: 10, failureTh
livenessProbe:   { httpGet: {path: /healthz, port: 8080}, periodSeconds: 20, failureTh
```

Design the endpoints deliberately: `/healthz` should answer "is this process alive and not deadlocked" with no external calls at all; `/readyz` may check that the database pool and model are loaded, because failing readiness only removes traffic. Conflating them is the single most common probe mistake.

HPA, PDB, and disruption

YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: rag-api, namespace: ragdemo }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: rag-api }
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
    - type: Pods
      pods:
        metric: { name: http_requests_inflight }
        target: { type: AverageValue, averageValue: "30" }
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies: [{ type: Percent, value: 100, periodSeconds: 30 }]
    scaleDown:
      stabilizationWindowSeconds: 300      # scale down slowly, avoid flapping
      policies: [{ type: Percent, value: 25, periodSeconds: 60 }]
---

apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: rag-api, namespace: ragdemo }
spec:
  minAvailable: 2                          # or maxUnavailable: 1
  selector:
    matchLabels: { app: rag-api }
```

HPA utilisation is calculated against the CPU **request**, not the limit — so a tiny request makes the HPA scale out constantly. A PDB constrains *voluntary* disruptions only: node drains, cluster upgrades, descheduler evictions. It does not protect against a node crashing. Set one for anything the customer considers production, or their next cluster upgrade will drain all your replicas simultaneously.

Placement: affinity, taints, tolerations

YAML

```
spec:
  nodeSelector: { "node.kubernetes.io/instance-type": "g5.xlarge" }
  tolerations:
    - key: "nvidia.com/gpu"
      operator: "Exists"
      effect: "NoSchedule"
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - { key: topology.kubernetes.io/zone, operator: In, values: [ap-south-1a
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          podAffinityTerm:
            topologyKey: kubernetes.io/hostname
            labelSelector:
              matchLabels: { app: rag-api }
```

A **taint** on a node repels pods; a **toleration** on a pod lets it ignore a specific taint. That is how GPU nodes, spot instances and dedicated node pools are reserved. Effects: `NoSchedule` (do not place new pods), `PreferNoSchedule` (soft), `NoExecute` (also evict pods already running). Anti-affinity on `kubernetes.io/hostname` spreads replicas across nodes so one node failure does not take the whole service down — use `preferred` rather than `required` on small clusters or your pods will sit Pending.

Configuration and Packaging: Helm and Kustomize

Raw YAML does not survive three environments. You need either templating (Helm) or patching (Kustomize). Customers usually have a strong existing preference; know both well enough to work in either.

Helm chart structure

TEXT

```
rag-api/
├── Chart.yaml          # name, version, appVersion, dependencies
├── values.yaml         # DEFAULTS - the chart's public interface
├── values.schema.json  # optional JSON Schema validation of values
├── templates/
│   ├── _helpers.tpl    # named template fragments (labels, names)
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── configmap.yaml
│   ├── serviceaccount.yaml
│   ├── hpa.yaml
│   ├── pdb.yaml
│   ├── NOTES.txt       # printed after install - put the access URL here
│   └── tests/
│       └── test-connection.yaml
└── charts/             # vendored sub-charts
```

YAML

```yaml
# values.yaml
replicaCount: 3
image:
  repository: registry.corp/platform/rag-api
  tag: ""                       # defaults to .Chart.AppVersion
  pullPolicy: IfNotPresent
imagePullSecrets: [{ name: corp-registry }]
resources:
  requests: { cpu: 250m, memory: 512Mi }
  limits:   { cpu: "2",  memory: 2Gi }
ingress:
  enabled: true
  className: nginx
  host: rag.apps.corp.example.com
  tls: true
config:
  logLevel: info
  topK: 8
externalSecrets:
  enabled: false
nodeSelector: {}
tolerations: []
affinity: {}
```

YAML

```
# templates/deployment.yaml (excerpt)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "rag-api.fullname" . }}
  labels: {{- include "rag-api.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels: {{- include "rag-api.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
      labels: {{- include "rag-api.selectorLabels" . | nindent 8 }}
    spec:
      {{- with .Values.imagePullSecrets }}
      imagePullSecrets: {{- toYaml . | nindent 8 }}
      {{- end }}
      serviceAccountName: {{ include "rag-api.serviceAccountName" . }}
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports: [{ name: http, containerPort: 8080 }]
          resources: {{- toYaml .Values.resources | nindent 12 }}
          env:
            - name: LOG_LEVEL
              value: {{ .Values.config.logLevel | quote }}
      {{- with .Values.nodeSelector }}
      nodeSelector: {{- toYaml . | nindent 8 }}
      {{- end }}
```

BASH

```
helm lint ./rag-api
helm template rag-api ./rag-api -f values-prod.yaml      # render locally, read the YA
helm install rag-api ./rag-api -n ragdemo --create-namespace -f values-prod.yaml
helm upgrade --install rag-api ./rag-api -n ragdemo -f values-prod.yaml \
  --set image.tag=1.4.3 --atomic --timeout 5m --wait
helm diff upgrade rag-api ./rag-api -f values-prod.yaml  # plugin; invaluable before p
helm history rag-api -n ragdemo
helm rollback rag-api 3 -n ragdemo
helm get values rag-api -n ragdemo                       # what is ACTUALLY deployed
helm get manifest rag-api -n ragdemo
helm uninstall rag-api -n ragdemo
helm package ./rag-api && helm push rag-api-0.3.1.tgz oci://registry.corp/charts
```

--atomic and --wait

Without them, a failed upgrade leaves the release half-applied and the customer looking at a broken service. `--atomic` rolls back automatically on failure; `--wait` blocks until resources are ready. Always use both for anything that is not a sandbox, and always run `helm diff upgrade` or `helm template | kubectl diff -f -` before touching a production namespace.

Kustomize

TEXT

```
base/
  kustomization.yaml
  deployment.yaml
  service.yaml
overlays/
  dev/   { kustomization.yaml, replica-patch.yaml }
  prod/  { kustomization.yaml, replica-patch.yaml, resources-patch.yaml }
```

YAML

```yaml
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: ragdemo
resources: [../../base]
commonLabels: { environment: prod }
images:
  - name: registry.corp/platform/rag-api
    newTag: 1.4.2
replicas:
  - { name: rag-api, count: 5 }
configMapGenerator:
  - name: rag-api-config
    literals: [log_level=warn, top_k=8]
patches:
  - path: resources-patch.yaml
    target: { kind: Deployment, name: rag-api }
```

BASH

```bash
kubectl kustomize overlays/prod          # render
kubectl apply -k overlays/prod
kubectl diff -k overlays/prod
```

	Helm	Kustomize
Mechanism	Go text templating + values	Strategic merge and JSON patches on plain YAML
Distribution	Versioned, packageable, an ecosystem of public charts	Plain directories in git
Release tracking	Built in: history, rollback, hooks	None; relies on git plus your GitOps tool
Readability	Templates become dense and hard to debug	Base files are valid YAML you can read directly
Built into kubectl	No, separate binary	Yes, `-k`
Use when	Shipping software to others; many knobs; third-party dependencies	Your own app across a few environments; GitOps with Argo CD or Flux

They compose: it is common to `helm template` a third-party chart and then post-process with Kustomize, and Argo CD supports exactly that pattern. For an FDE shipping a product into many customers, Helm is usually the right answer because the chart *is* the configuration contract you hand over.

Security in a Customer Cluster

You are a guest in a cluster someone else is accountable for. Everything you deploy will be reviewed against their policy. Knowing the controls in advance means you write compliant manifests the first time instead of iterating through rejections for a week.

RBAC

YAML

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: rag-api
  namespace: ragdemo
automountServiceAccountToken: false      # the app does not call the API - do not hand
---

apiVersion: rbac.authorization.k8s.io/v1
kind: Role                               # Role = namespaced; ClusterRole = cluster-w
metadata: { name: rag-api-reader, namespace: ragdemo }
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    resourceNames: ["rag-api-config"]     # narrow to the exact object
    verbs: ["get", "watch", "list"]
---

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: rag-api-reader, namespace: ragdemo }
subjects:
  - { kind: ServiceAccount, name: rag-api, namespace: ragdemo }
roleRef: { kind: Role, name: rag-api-reader, apiGroup: rbac.authorization.k8s.io }
```

BASH

```
kubectl auth can-i --list --as=system:serviceaccount:ragdemo:rag-api -n ragdemo
kubectl auth can-i delete pods --as=system:serviceaccount:ragdemo:rag-api -n ragdemo
kubectl get rolebinding,clusterrolebinding -A -o json | jq -r '
  .items[] | select(.roleRef.name=="cluster-admin")
  | "\(.kind) \(.metadata.name) → \([.subjects[]?.name] | join(","))"'
```

RBAC is purely additive — there are no deny rules. Permissions are the union of every binding that applies to you. The audit question to ask any customer cluster is "who is bound to cluster-admin", and the answer is frequently longer than it should be.

Pod Security Admission

PodSecurityPolicy was removed in 1.25. Its replacement, Pod Security Admission, is a namespace label enforcing one of three profiles.

Profile	Allows	Blocks
`privileged`	Everything	Nothing — for system namespaces only
`baseline`	Most normal workloads	Privileged containers, host namespaces, hostPath, most capabilities
`restricted`	Hardened workloads only	Also requires `runAsNonRoot`, `allowPrivilegeEscalation: false`, `drop: ALL`, `seccompProfile: RuntimeDefault`, no hostPorts

YAML

```yaml
# A pod spec that passes "restricted" - memorise this block
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  fsGroup: 10001
  seccompProfile: { type: RuntimeDefault }
containers:
  - name: api
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: { drop: ["ALL"] }
```

`readOnlyRootFilesystem: true` is the one that breaks working applications: anything writing to `/tmp`, a cache directory or a log file now fails. The fix is an `emptyDir` mounted at each writable path,

which is better practice anyway. Many customers also run OPA Gatekeeper or Kyverno with additional policies — ask for the policy set on day one and test against it with `kubectl apply --dry-run=server`.

Network policies

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-all, namespace: ragdemo }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: rag-api-allow, namespace: ragdemo }
spec:
  podSelector:
    matchLabels: { app: rag-api }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: ingress-ngi
      ports: [{ protocol: TCP, port: 8080 }]
  egress:
    - to: [{ podSelector: { matchLabels: { app: postgres } } }]
      ports: [{ protocol: TCP, port: 5432 }]
    - to:                                  # CoreDNS - forget this and everything bre
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system
          podSelector: { matchLabels: { k8s-app: kube-dns } }
      ports: [{ protocol: UDP, port: 53 }, { protocol: TCP, port: 53 }]
```

Default-deny without a DNS rule breaks everything

Applying a default-deny egress policy and forgetting UDP/TCP 53 to kube-dns produces a cluster where every pod can reach nothing and the error messages all look like application bugs. Add the DNS rule in the same commit as the deny rule, every time. Also check the CNI actually enforces NetworkPolicy — Calico, Cilium and Weave do; the plain AWS VPC CNI without add-ons silently ignores them, so your policies look applied and enforce nothing.

Image pull secrets and private registries

BASH

```bash
kubectl create secret docker-registry corp-registry \
  --docker-server=registry.corp.example.com:5000 \
  --docker-username=svc_k8s --docker-password="$REG_PASS" \
  --docker-email=noreply@corp.example.com -n ragdemo

# Attach to a ServiceAccount so every pod using it inherits the secret
kubectl patch serviceaccount rag-api -n ragdemo \
  -p '{"imagePullSecrets":[{"name":"corp-registry"}]}'
```

Real secrets management

YAML

```yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: rag-api-secrets, namespace: ragdemo }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: corp-vault, kind: ClusterSecretStore }
  target:
    name: rag-api-secrets
    creationPolicy: Owner
  data:
    - secretKey: db_password
      remoteRef: { key: kv/ragdemo/postgres, property: password }
    - secretKey: openai_api_key
      remoteRef: { key: kv/ragdemo/llm, property: api_key }
```

EXTERNAL SECRETS OPERATOR

Syncs from Vault, AWS Secrets Manager, Azure Key Vault or GCP SM into a native Secret. Git holds only a reference. The most common choice at enterprises today.

SECRETS STORE CSI DRIVER

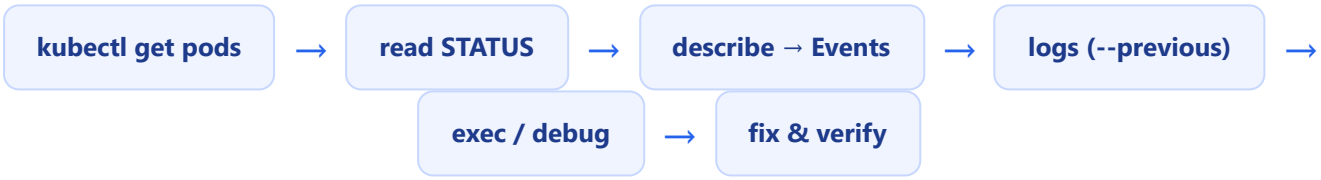
Mounts secrets as files straight from the external store — the value never becomes a Kubernetes Secret object at all. Stronger, slightly more friction.

SEALED SECRETS

Encrypt with a cluster-held public key; the ciphertext is safe in git. Simple and self-contained, but rotation and key management are on you.

Debugging a Broken Pod

This is the runbook. Start at the Pod's status string and follow the branch — almost every Kubernetes failure resolves in under five minutes with this sequence.



ImagePullBackOff / ErrImagePull

`describe` shows `Failed to pull image … rpc error`. Read the exact error text — it names the cause.

Error fragment	Cause	Fix
`manifest unknown` / `not found`	Wrong tag or repo path	Verify with `crane ls` or `skopeo inspect`; check for a typo and for the registry path prefix the customer requires
`unauthorized` / `authentication required`	Missing or wrong imagePullSecret	Check it exists in the *same namespace* and is attached to the pod or its ServiceAccount
`x509: certificate signed by unknown authority`	Registry uses a corporate CA the nodes do not trust	CA bundle must be installed on every **node**, in containerd's config — not in your pod
`dial tcp … i/o timeout`	Nodes have no route or egress to the registry	Air-gapped cluster: mirror the image internally first
`toomanyrequests`	Docker Hub anonymous rate limit	Authenticate, or mirror to the internal registry
`no match for platform`	arm64 image, amd64 nodes	Rebuild with buildx for the right platform

BASH

```
kubectl describe pod rag-api-xk2mq | grep -A5 Events
kubectl get secret corp-registry -n ragdemo -o jsonpath='{.data.\.dockerconfigjson}' |
kubectl get pod rag-api-xk2mq -o jsonpath='{.spec.imagePullSecrets}'
```

CrashLoopBackOff

The container starts and exits repeatedly; the kubelet backs off exponentially (10 s, 20 s, 40 s … capped at 5 min). CrashLoopBackOff is a *symptom*, never a cause — the cause is in the previous container's output.

BASH

```
kubectl logs rag-api-xk2mq --previous          # THE command for this state
kubectl get pod rag-api-xk2mq -o jsonpath='{.status.containerStatuses[0].lastState.ter
# { "exitCode": 137, "reason": "OOMKilled", "startedAt": "…", "finishedAt": "…" }
kubectl describe pod rag-api-xk2mq | grep -E 'Last State|Exit Code|Reason|Restart Coun
```

- [ ] Exit 1 with a stack trace — application config error: missing env var, bad DSN, unreachable dependency
- [ ] Exit 137 with `OOMKilled` — memory limit too low, or a real leak. Raise the limit once to confirm, then investigate properly
- [ ] Exit 127 — command not found; wrong path or a distroless image with no shell for a shell-form command
- [ ] Exit 126 — not executable; missing `chmod +x` or CRLF line endings in an entrypoint script
- [ ] Exit 0 immediately — your command finished; the process daemonised itself or the CMD is wrong
- [ ] No logs at all — the failure is before your code runs: check `describe` for mount and secret errors

BASH

```
# Keep it alive long enough to look around
kubectl run debug --image=registry.corp/platform/rag-api:1.4.2 --restart=Never \
  --command -- sleep 3600
kubectl exec -it debug -- sh
# Or, for a distroless image with no shell, attach an ephemeral container:
kubectl debug -it rag-api-xk2mq --image=busybox:1.36 --target=api -- sh
```

Pending

BASH

```
kubectl describe pod rag-api-xk2mq | tail -20
# Warning  FailedScheduling  0/5 nodes are available:
#   2 Insufficient cpu, 1 node(s) had untolerated taint {gpu: true},
#   2 node(s) didn't match pod anti-affinity rules.
```

- [ ] `Insufficient cpu/memory` — requests exceed anything free. Check `kubectl describe node | grep -A8 "Allocated resources"`; lower requests or add capacity
- [ ] `untolerated taint` — add a toleration, or target a different node pool
- [ ] `didn't match node selector / affinity` — your constraint cannot be satisfied; check node labels

- [ ] `had volume node affinity conflict` — the PV is in zone A, the pod is being placed in zone B
- [ ] `pod has unbound immediate PersistentVolumeClaims` — no matching StorageClass, or the provisioner is broken. `kubectl get pvc,sc`
- [ ] Pending with **no** events — exceeded a ResourceQuota, or an admission webhook is failing. `kubectl get events -n ragdemo`

Running but not Ready

BASH

```
kubectl get pods            # 0/1 Running — the container is up, readiness is failing
kubectl describe pod rag-api-xk2mq | grep -A3 Readiness
# Warning  Unhealthy  Readiness probe failed: Get "http://10.244.2.7:8080/readyz":
#                     dial tcp 10.244.2.7:8080: connect: connection refused
kubectl exec rag-api-xk2mq -- wget -qO- http://127.0.0.1:8080/readyz
kubectl exec rag-api-xk2mq -- ss -tulpn
```

Connection refused from the probe but success from inside usually means the app bound `127.0.0.1` instead of `0.0.0.0` — the kubelet probes the Pod IP, not loopback. Other causes: the probe port does not match the container port, the path returns a non-2xx, `initialDelaySeconds` is too short for startup, or the readiness endpoint checks a dependency that is genuinely down.

503 through the Ingress

1. **Does the Service have endpoints?**
   `kubectl get endpoints rag-api -n ragdemo`. Empty means the Service selector does not match any *ready* Pod labels. Compare `kubectl get svc rag-api -o jsonpath='{.spec.selector}'` with `kubectl get pods --show-labels`. This is the cause more often than anything else.
2. **Does the Pod work directly?**
   `kubectl port-forward pod/rag-api-xk2mq 8080:8080` then `curl localhost:8080/healthz`. If this fails, the problem is the app, not the network.
3. **Does the Service work?**
   `kubectl port-forward svc/rag-api 8080:80`. If the Pod works but the Service does not, check `targetPort` against the actual container port.
4. **Does in-cluster DNS resolve?**

`kubectl run t --rm -it --image=nicolaka/netshoot --restart=Never -- sh`, then `dig rag-api.ragdemo.svc.cluster.local` and `curl http://rag-api.ragdemo/healthz`.

5 **Is the Ingress wired up?**
`kubectl describe ingress rag` — check the backend shows real endpoint IPs, not `<none>`, and that `ingressClassName` matches a controller that exists.

6 **Read the controller's own logs**
`kubectl logs -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx --tail=100`. Upstream connect errors, TLS handshake failures and the exact upstream address all appear here.

7 **Check NetworkPolicy**
A default-deny in the namespace without an allow rule from the ingress-controller namespace produces exactly this symptom, with timeouts rather than refusals.

Node-level problems

BASH

```
kubectl get nodes
kubectl describe node ip-10-0-3-21 | grep -A10 Conditions
# MemoryPressure  False / DiskPressure True / PIDPressure False / Ready True
kubectl get events -A --field-selector reason=Evicted
kubectl get pods -A -o wide --field-selector spec.nodeName=ip-10-0-3-21
kubectl cordon ip-10-0-3-21                                    # stop new scheduling
kubectl drain ip-10-0-3-21 --ignore-daemonsets --delete-emptydir-data --grace-period=6
kubectl uncordon ip-10-0-3-21
```

`DiskPressure: True` triggers image garbage collection and then pod eviction — usually caused by image sprawl or a pod writing unbounded data to an `emptyDir`. `NotReady` normally means the kubelet stopped reporting: check `systemctl status kubelet` and `journalctl -u kubelet -n 200` on the node itself if you have access.

Deploying Into a Customer's Environment

The technical work is often the easy half. Getting your image through an air gap, past a security review, and running inside a policy-enforced namespace with no egress is the part that determines whether the engagement lands on time.

Air-gapped registries

1. **Enumerate every image**
   Not just yours — Postgres, Redis, the vector store, ingress-nginx, cert-manager, the metrics server, every sidecar, and every init container. `helm template . | grep -Eo 'image: .*' | sort -u` is the starting point, and `kubectl get pods -A -o jsonpath='{..image}' | tr ' ' '\n' | sort -u` catches what you missed.

2. **Pin to digests**
   Resolve every tag to a digest before you mirror, so what you validated is provably what ships.

3. **Mirror**
   `skopeo copy --all docker://upstream/img:tag docker://registry.corp/mirror/img:tag` — `--all` preserves the multi-arch manifest list. For a true air gap, `skopeo sync --src docker --dest dir` to a directory, carry it across, then `skopeo sync --src dir --dest docker`.

4. **Rewrite references**
   Helm values with an `image.registry` prefix, or a Kustomize `images:` block, or a cluster-wide `ImageContentSourcePolicy` / containerd mirror config so the rewrite happens at the node.

5. **Scan on the inside**
   Run Trivy with a pre-downloaded vulnerability database (`trivy --cache-dir ./trivy-cache image --skip-db-update`) — the scanner itself needs updates it cannot fetch.

6. **Document the provenance**
   Source image, digest, scan date, SBOM, and who approved it. The customer's auditors will ask, possibly months later.

BASH

```
helm template rag ./rag-api -f values-prod.yaml \
  | grep -Eo 'image: *"?[^"]+' | awk '{print $2}' | tr -d '"' | sort -u

for img in $(cat images.txt); do
  skopeo copy --all --retry-times 3 "docker://$img" \
    "docker://registry.corp/mirror/${img#*/}" || echo "FAILED $img" >> failures.txt
done
```

Private CA certificates

YAML

```yaml
# Distribute the corporate root to every pod that makes outbound TLS calls
apiVersion: v1
kind: ConfigMap
metadata: { name: corp-ca-bundle, namespace: ragdemo }
data:
  corp-root.pem: |
    -----BEGIN CERTIFICATE-----
    MIIF...
    -----END CERTIFICATE-----
---

# in the pod spec
volumeMounts:
  - { name: corp-ca, mountPath: /etc/ssl/certs/corp-root.pem, subPath: corp-root.pem,
env:
  - { name: REQUESTS_CA_BUNDLE,  value: /etc/ssl/certs/corp-root.pem }   # Python requ
  - { name: SSL_CERT_FILE,       value: /etc/ssl/certs/corp-root.pem }   # OpenSSL, Go
  - { name: NODE_EXTRA_CA_CERTS, value: /etc/ssl/certs/corp-root.pem }   # Node.js
  - { name: CURL_CA_BUNDLE,      value: /etc/ssl/certs/corp-root.pem }
```

Java is the exception — it uses its own truststore, so you need an init container running `keytool -importcert -cacerts -storepass changeit -noprompt -alias corp -file /ca/corp-root.pem`, or a base image with the cert already imported. The registry CA is separate again: that must be trusted by containerd on each node, which is a cluster-admin change, not something you can do from a manifest.

Egress restrictions

YAML

```yaml
env:
  - name: HTTPS_PROXY
    value: http://proxy.corp.example.com:3128
  - name: HTTP_PROXY
    value: http://proxy.corp.example.com:3128
  - name: NO_PROXY
    value: "localhost,127.0.0.1,.svc,.svc.cluster.local,.cluster.local,10.0.0.0/8,169.
```

The NO_PROXY list is not optional

Omit `.svc` and `.cluster.local` and every in-cluster call — to Postgres, to the vector store, to the Kubernetes API — is routed to a corporate proxy that cannot resolve cluster-internal names. The symptom is a 30-second hang followed by a connection error on calls that obviously should work locally. Omit `169.254.169.254` and cloud credential fetching (IRSA, workload identity) breaks. Set both upper- and lowercase variants, because libraries disagree about which they read.

The pre-flight checklist before you ask for a deployment window

- [ ] Image mirrored to the internal registry, pinned by digest, scanned, SBOM attached
- [ ] Runs as a numeric non-root UID with `allowPrivilegeEscalation: false` and `drop: ALL`
- [ ] `readOnlyRootFilesystem: true` with `emptyDir` volumes for every writable path
- [ ] CPU and memory requests *and* limits set on every container — a ResourceQuota will reject you otherwise
- [ ] Startup, readiness and liveness probes defined, with the startup probe sized for the real cold-start time
- [ ] PodDisruptionBudget and anti-affinity so the next cluster upgrade does not take you down
- [ ] NetworkPolicy written, including the DNS egress rule, and tested in a namespace with default-deny
- [ ] Secrets sourced from the customer's vault, not from a manifest in git
- [ ] Corporate CA mounted and the right per-runtime env var set
- [ ] Proxy variables set with a complete `NO_PROXY`
- [ ] Manifests render cleanly through their policy engine: `kubectl apply --dry-run=server`
- [ ] Rollback path rehearsed: `helm rollback` or `kubectl rollout undo`, tested in staging
- [ ] Logs go to stdout in the format their aggregation stack expects; metrics exposed on a scrapeable port
- [ ] A one-page runbook: what it is, what it depends on, how to restart it, who to call

✓ **Start the approval process on day one**

At a regulated customer, image approval, firewall exceptions and namespace provisioning routinely take two to three weeks of calendar time while taking two hours of actual work. Submit the request before you have finished building, with a placeholder image if necessary. The engineers who deliver on time are usually the ones who started the paperwork early, not the ones who wrote better YAML.

Interview Questions

These come up constantly in FDE, platform and SRE loops. Answer with the mechanism, then the operational consequence — that combination is what distinguishes someone who has run this in production.

What is a container, really? How is it different from a VM?

A container is a normal Linux process that the kernel has placed into a set of namespaces and cgroups. Namespaces control what it can see — PID namespace so it has its own process tree with your app as PID 1, mount namespace for its own filesystem view, network namespace for its own interfaces and port space, plus UTS, IPC and user namespaces. cgroups control what it can consume — CPU quota, memory ceiling, I/O weight, PID count. The filesystem comes from stacked read-only image layers plus a thin writable layer, usually via overlayfs. A VM, by contrast, virtualises hardware and boots a complete guest kernel on a hypervisor. The practical differences follow directly: containers start in under a second and cost megabytes because there is no kernel to boot; they share the host kernel, so a Linux container cannot run natively on Windows and a kernel exploit from inside a container compromises the host. That last point is why customers care about running as non-root, dropping capabilities, and seccomp profiles — and why workloads with hard multi-tenancy requirements sometimes use Kata Containers or Firecracker to get a lightweight VM boundary back.

Your image is 1.4 GB. How do you get it under 200 MB?

First I measure where it actually went with `docker history` and `dive`, because guessing wastes time. Then, in rough order of impact: introduce a multi-stage build so compilers and dev headers — build-essential, gcc, libpq-dev, the full Node toolchain — exist only in the builder stage and never reach the runtime image; that is typically 400–600 MB on its own. Switch the final stage to a slim or distroless base. Collapse package installation into a single `RUN` with `--no-install-recommends` and `rm -rf /var/lib/apt/lists/*` in the same layer, since cleanup in a later layer frees nothing. Add a proper `.dockerignore` so `.git`, `node_modules`, test fixtures and local virtualenvs never enter the build context. For Node, `npm prune --omit=dev` after the build; for Python, install into a venv in the builder and copy only the venv. I would be cautious about jumping to Alpine for a Python service — musl means no manylinux wheels, so pip compiles from source and the image can end up larger and far slower to build. And I would verify the result with a CVE scan, because the size reduction

usually drops the vulnerability count by an order of magnitude too, which is what the customer's security review actually cares about.

Walk me through what happens when you run `kubectl apply -f deployment.yaml`.

kubectl reads the file, merges it with the previous applied configuration annotation, and sends a request to the kube-apiserver. The apiserver authenticates the caller, authorises via RBAC, runs mutating admission webhooks (which might inject a sidecar or defaults), validates the object against the schema, runs validating admission webhooks and Pod Security Admission, then persists it to etcd. The Deployment controller is watching Deployments; it sees the new spec and creates or updates a ReplicaSet with a pod-template hash. The ReplicaSet controller sees a replica count it has not satisfied and creates Pod objects with `nodeName` unset. The scheduler watches for unscheduled Pods, filters nodes by resource requests, taints, affinity and volume topology, scores the survivors, and binds the Pod to a node. The kubelet on that node sees a Pod bound to it, pulls the image through containerd, sets up the sandbox and CNI networking, mounts volumes, starts the containers, and begins running probes. Once readiness passes, the endpoints controller adds the Pod IP to the Service's EndpointSlice, kube-proxy programs iptables or IPVS rules on every node, and traffic starts flowing. Every one of those stages is a place it can get stuck, which is exactly why `kubectl describe` and the Events list are the first things I read when a Pod is not where I expect it.

A pod is in CrashLoopBackOff. Diagnose it.

CrashLoopBackOff is never the cause, only the kubelet's exponential backoff after repeated exits. The first command is `kubectl logs <pod> --previous`, because plain `logs` shows the current container which may have just started and printed nothing. In parallel I read `kubectl get pod -o jsonpath='{.status.containerStatuses[0].lastState.terminated}'` for the exit code and reason. Exit 1 with a stack trace is an application or configuration failure — a missing environment variable, a bad connection string, a dependency that is not up. Exit 137 with reason OOMKilled means it exceeded its memory limit; I raise it once to confirm the diagnosis and then look at whether it is a genuine leak or just a limit set from a guess. Exit 127 is command not found — often a shell-form command in a distroless image. Exit 126 is not executable, frequently CRLF line endings in an entrypoint script written on Windows. Exit 0 immediately means the process completed, so the CMD is wrong or the app daemonised itself. If there are no logs at all, the failure is before my code runs, so I go to `describe` and look for mount failures, missing secrets or ConfigMaps. To get hands-on I either run the image with `--command -- sleep 3600` so it stays up, or attach an ephemeral debug container with `kubectl debug --target`, which works even on a distroless image with no shell.

Explain requests versus limits, and what happens when each is exceeded.

Requests are what the scheduler uses to decide whether a Pod fits on a node; they are a reservation and the node's allocatable capacity is divided among them. Limits are enforced at runtime by the kernel through cgroups. The two resources behave completely differently. CPU is compressible: exceeding the limit causes CFS throttling — the process is frozen for the remainder of each 100 ms period — so the symptom is latency, not failure, and it is invisible in average CPU utilisation. Memory is incompressible: exceeding the limit means an immediate OOM kill with exit code 137 and no

chance to clean up. The combination also determines QoS class: requests equal to limits on every container gives Guaranteed, which is evicted last under node pressure; some requests set gives Burstable; nothing set gives BestEffort, which is evicted first. My practical defaults are memory request equal to memory limit so the pod is predictable and never surprises the node, a CPU request based on measured steady-state usage, and a CPU limit that is either generous or absent for latency-sensitive services because throttling causes p99 spikes that customers notice. I also check whether the runtime honours the cgroup — an older JVM or a library sizing thread pools from `os.cpu_count()` will read host values and badly over-provision inside a small limit.

Difference between liveness, readiness and startup probes?

Readiness controls traffic: if it fails, the Pod is removed from the Service's endpoints and receives no requests, but it is not restarted. Liveness controls lifecycle: if it fails, the kubelet kills and restarts the container. Startup gates the other two: while it is running, liveness and readiness are suspended, which is how you protect a slow-starting application without having to set a huge `initialDelaySeconds` on liveness. The classic production incident comes from pointing a liveness probe at an endpoint that checks the database — when the database has a brief hiccup, every replica fails liveness simultaneously, all restart, the cold-start thundering herd hits the database again, and a five-second blip becomes a twenty-minute outage. So liveness should test only "is this process alive and not deadlocked", with no external dependencies; readiness is where it is safe to check dependencies, because the only consequence is traffic shedding. I also make sure readiness fails during shutdown and pair it with a short `preStop` sleep, since endpoint removal propagates asynchronously and without that gap you get a burst of 502s on every deploy.

How does a request get from an external user to a container?

DNS resolves the hostname to the external IP of a cloud load balancer or the node IPs behind a DNS record. The load balancer forwards to a NodePort on some node, where the ingress controller's Service is listening. The ingress controller — nginx, Traefik, an ALB controller — terminates TLS, matches the Host header and path against Ingress rules, and selects a backend Service. Importantly, modern ingress controllers do not route through the Service's cluster IP; they watch EndpointSlices and proxy straight to a Pod IP, which is why they can do their own load balancing and session affinity. If traffic does go via a ClusterIP, kube-proxy has programmed iptables or IPVS rules on every node that DNAT the virtual IP to one of the ready backend Pod IPs. The packet then traverses the CNI — a veth pair into the Pod's network namespace, possibly via an overlay like VXLAN, possibly native VPC routing — and arrives at the container's listening socket. For debugging I walk that chain backwards: check endpoints exist, port-forward to the Pod, then to the Service, then test DNS from inside the cluster, then read the ingress controller logs. The layer where it stops failing tells you exactly which component is broken.

Helm or Kustomize?

They solve adjacent problems. Helm is templating plus release management: it renders Go templates against a values file and tracks installed releases so you get `helm history`, `helm rollback`, hooks and atomic upgrades. Kustomize is patching: the base files are valid YAML you can read and `kubectl`

`apply` directly, and overlays apply strategic merge patches on top. My rule is that if I am distributing software to other people — which as an FDE I usually am — Helm wins, because the chart's `values.yaml` is a documented configuration contract and the customer can version, review and upgrade it like any other dependency. If I am managing my own application across a few environments in a GitOps repo, Kustomize is less machinery and the diffs are readable. They also compose: Argo CD can render a Helm chart and then apply Kustomize patches, which is how teams customise a third-party chart without forking it. The practical warnings are that Helm templates become genuinely painful to debug once they grow conditionals — `helm template` and `helm diff upgrade` are non-negotiable before any production apply — and that Kustomize alone has no rollback story, so you are relying on git and your deployment tool for that.

How do you deploy into an air-gapped customer cluster?

The first step is enumerating every image, not just mine — the database, cache, vector store, ingress controller, cert-manager, metrics-server, every sidecar and init container. `helm template | grep image:` gets most of it. I resolve each tag to a digest so what I validated is provably what ships, then mirror with `skopeo copy --all` to preserve multi-arch manifests, or `skopeo sync` to a directory for physical transfer across the gap. On the inside I rewrite references through Helm values with a registry prefix, a Kustomize `images:` block, or a node-level containerd mirror so the rewrite is transparent. Three things routinely bite people: the registry's TLS certificate is signed by the corporate CA, which must be trusted by containerd on every node — that is a cluster-admin change, not something a manifest can fix; vulnerability scanners need their database pre-staged because they cannot phone home; and any Helm chart that pulls a dependency at install time will fail, so charts must be fully vendored. I also budget calendar time deliberately — image approval and firewall exceptions at a regulated customer are routinely two to three weeks of waiting for two hours of work, so I start that process on day one with a placeholder if necessary.

What is a Service, and why does my pod get "connection refused" calling another service?

A Service is a stable virtual IP and DNS name that load-balances across the set of Pods matching its label selector and currently passing readiness. The backing implementation is an EndpointSlice maintained by the endpoints controller plus iptables or IPVS rules programmed by kube-proxy on every node. For "connection refused" I check four things in order. First, does the Service have endpoints at all — `kubectl get endpoints`; empty is the most common cause and means the selector does not match any ready Pod's labels, usually because someone matched the Deployment name instead of the pod template labels, or because every Pod is failing readiness. Second, is the name right — cross-namespace calls need `service.namespace` or the FQDN, and using the bare name from another namespace fails silently. Third, is the port right — the Service `port` is what clients dial and `targetPort` is the container port; mixing them up gives exactly this error. Fourth, is the application bound to `0.0.0.0` rather than `127.0.0.1` inside the container, which I verify with `kubectl exec -- ss -tulpn`. If all four are fine I look at NetworkPolicy — but note that a policy drop usually produces a timeout rather than a refusal, so the error text itself narrows the search.

What does `readOnlyRootFilesystem: true` break, and how do you fix it?

> It breaks anything that writes outside an explicitly mounted volume: temp files in `/tmp`, a framework's cache directory, a log file written to disk, a PID or lock file, a Python `__pycache__`, a Node package cache, SQLite's journal. The fix is not to turn it off; it is to mount an `emptyDir` at each path that genuinely needs to be writable, with a `sizeLimit` so a runaway write cannot trigger DiskPressure on the node and evict every pod. I find the paths empirically — run the container, watch it fail, add a mount, repeat, or use `docker diff` on a locally running container which lists exactly which files were modified relative to the image. Applications should also be configured to log to stdout rather than a file, which is what the cluster's log aggregation expects anyway. It is worth doing properly because `readOnlyRootFilesystem` is part of the `restricted` Pod Security profile that most enterprise clusters enforce, so if I do not solve it at build time the customer's admission controller will reject my deployment and I will be solving it under time pressure instead.

> **When would you tell a customer not to use Kubernetes?**
>
> When the operational cost exceeds the benefit, which is more often than the industry admits. If they are running one or two services with predictable load on a couple of VMs, systemd units with a reverse proxy, or Docker Compose on a single host, delivers the same outcome with a fraction of the surface area to secure, upgrade and debug. If they have no platform team and no on-call rotation, handing them a cluster means handing them a system nobody can operate once I leave — cluster upgrades, certificate rotation, CNI troubleshooting and etcd maintenance are real recurring work. If their workload is genuinely serverless-shaped, a managed container service or a function platform removes the cluster entirely. I would push toward Kubernetes when they need multi-service orchestration with real service discovery, horizontal autoscaling, zero-downtime rolling deploys with health gating, self-healing across node failures, or when they already run a cluster and adding my workload is marginal cost rather than new cost. The honest version of this conversation is also good consulting: recommending less infrastructure than the customer expected builds the credibility you need for the recommendations where you do say yes.

Hands-On Lab Plan

Nothing here is learned by reading. Work through these on a local cluster — kind, k3d or minikube are all sufficient, and `kind create cluster --config kind.yaml` with three nodes gives you realistic scheduling behaviour on a laptop.

Setup

BASH

```
kind create cluster --name lab --config - <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
    extraPortMappings:
      - { containerPort: 80,  hostPort: 8080, protocol: TCP }
      - { containerPort: 443, hostPort: 8443, protocol: TCP }
  - role: worker
  - role: worker
EOF

kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deplo
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/dow
```

Week 1 — Docker

Day	Build this	Prove it
1	A naive single-stage Dockerfile for a FastAPI app	Record the size and the rebuild time after a one-character source change
2	Reorder for cache; add `.dockerignore`	Rebuild time drops from minutes to seconds. Record both numbers
3	Convert to multi-stage with a slim runtime and a non-root numeric UID	Image under 200 MB; `docker run` shows `uid=10001`
4	Add healthcheck, resource limits, read-only rootfs with tmpfs	`docker inspect` shows healthy; the app still writes to `/tmp`
5	Scan with Trivy, generate an SBOM, fix the top findings	Zero fixable CRITICAL; SBOM file produced
6	The full Compose stack: API + Postgres + Qdrant + Redis + UI	`docker compose up -d` from a clean machine, end-to-end request works
7	Break it deliberately: wrong port, wrong network, missing env, OOM limit	Diagnose each from the error message alone, under two minutes

Week 2 — Kubernetes

Day	Build this	Prove it
8	Deployment + Service + ConfigMap + Secret for the API	`kubectl port-forward svc/…` returns 200
9	Ingress with a host rule; the UI and API on different paths	`curl -H 'Host: rag.local' localhost:8080/api/healthz`
10	StatefulSet + PVC for Postgres; an init container that waits for it	Delete the Pod; data survives; the API reconnects
11	Probes, requests/limits, PDB, anti-affinity, HPA	Load-test until the HPA scales; `kubectl drain` respects the PDB
12	Package it as a Helm chart with dev and prod values	`helm upgrade --install --atomic`, then `helm rollback` successfully
13	RBAC, a restricted-profile namespace, default-deny NetworkPolicy	Everything still works after the policies are applied
14	Air-gap simulation: a local registry, no internet, mirror everything	Full install on a cluster with egress blocked

Deliberate failure drills

Drill 1 — Six broken pods

Drill 2 — Zero-downtime rollout

Drill 3 — Air-gapped install, end to end

Drill 4 — Resource exhaustion

✓ **The bar to aim for**

Given a customer cluster you have never seen, a repository you did not write and two hours before a demo, you should be able to build an image that passes their security policy, deploy it into a namespace with quotas and default-deny networking, expose it through their ingress, and diagnose anything that goes wrong from the Pod status string without reaching for a search engine. Everything in this guide exists to make that two-hour window comfortable.
